

FocusTracer Python Test Süreç Senaryoları

Execution Trace Doğrulama, Standartlaştırma ve Ayırıştırma Modülleri İçin Test Paketi

Doküman Amacı: FocusTracer yazılımının parser, log doğrulama (validation) ve trace standartlaştırma mekanizmalarını test etmek amacıyla kurgulanmış 15 adet Python senaryosudur. Paket; mantıksal hatalar (logical), semantik/çalışma zamanı hataları (semantic/runtime) ve referans alınabilecek hatasız (clean) kod blokları içermektedir. Hatalı durumlar her senaryonun altında özel olarak bildirilmiştir.

Test Senaryoları Listesi

Senaryo 1: Sipariş Toplamı Hesaplama

MANTIKSAL HATA (LOGICAL BUG)

```
def calculate_total_price(cart_items, discount_threshold=100):
    total = 0
    for item in cart_items:
        if item['price'] < 0:
            continue
        total += item['price']

    # HATA: > yerine < kullanılmış. Eşik değerinin altındakilere indirim uygulanıyor.
    if total < discount_threshold:
        print("Tebrikler! %10 indirim kazandınız.")
        total = total * 0.90

    return total

my_cart = [{'name': 'Kitap', 'price': 40}, {'name': 'Kulaklık', 'price': 80}]
final_amount = calculate_total_price(my_cart)
print(f"Ödenecek Tutar: {final_amount}")
```

FOCUSTRACER DOĞRULAMA RAPORU VE BEKLENTİ:

if total < discount_threshold: satırında mantık hatası vardır. Sepet tutarı 120 TL olmasına rağmen indirim uygulanmaz, aksine 100 TL altı sepetlerde indirim tetiklenir. Trace loglarında if koşulunun beklenenin aksine True/False dönmesi incelenmelidir.

Senaryo 2: Kullanıcı Profil Güncelleme

SEMANTİK / ÇALIŞMA ZAMANI HATASI (SEMANTIC / RUNTIME BUG)

```
user_db = {"user_123": {"name": "Ahmet", "role": "developer"}}

def update_user_role(user_id, new_role):
    if user_id in user_db:
        # HATA: Sözlük yapısına liste indeksiyle (0) erişilmeye çalışılıyor.
        user_data = user_db[0]
        user_data['role'] = new_role
    else:
        print("Kullanıcı bulunamadı.")

update_user_role("user_123", "lead_developer")
```

FOCUSRACER DOĞRULAMA RAPORU VE BEKLENTİ:

user_db[0] satırı semantik olarak KeyError hatası fırlatır çünkü user_db bir dictionary'dir ve 0 anahtarına sahip değildir. FocusTracer izleme logunda execution'ın bu satırda kesilip Exception fırlattığı doğrulanmalıdır.

Senaryo 3: Sıcaklık Değerleri Ortalaması

HATASIZ (CLEAN CODE / BASELINE)

```
def get_average_temperature(readings):
    if not readings:
        return 0
    valid_readings = [t for t in readings if -50 <= t <= 60]
    if len(valid_readings) == 0:
        return 0
    return sum(valid_readings) / len(valid_readings)

sensor_data = [22.5, 23.0, 100.0, 21.8, -10.0]
avg_temp = get_average_temperature(sensor_data)
print(f"Ortalama Sıcaklık: {avg_temp}")
```

FOCUSRACER DOĞRULAMA RAPORU VE BEKLENTİ:

Kod tamamen hatasızdır. Uç değer olan 100.0 elenir ve doğru ortalama hesaplanır. FocusTracer'ın başarılı doğrusal akış senaryolarını, döngüleri ve liste filtrelemelerini nasıl trace ettiğini doğrulamak için referans kodudur.

Senaryo 4: Banka Hesabı Para Çekme Kontrolü

MANTIKSAL HATA (LOGICAL BUG)

```
def withdraw_money(account, amount):  
    # HATA: Bakiye kontrolü ters yapılmış. Hesapta para varsa çekim engelleniyor!  
    if account['balance'] >= amount:  
        print("Yetersiz bakiye!")  
        return account['balance']  
  
    account['balance'] -= amount  
    return account['balance']  
  
my_account = {'owner': 'Sergen', 'balance': 500}  
withdraw_money(my_account, 150)
```



FOCUSTRACER DOĞRULAMA RAPORU VE BEKLENTİ:

if account['balance'] >= amount: ifadesi mantıksal olarak hatalıdır. Bakiye miktardan büyük veya eşitken yetersiz bakiye uyarısı verir. Trace loglarında if bloğunun içine girilme akışı ve değişken mutasyonu incelenmelidir.

Senaryo 5: Veri Tabanı Bağlantı Havuzu (Connection Pool)

SEMANTİK / KAYNAK SIZINTISI HATASI (SEMANTIC / RESOURCE LEAK)

```
class DBConnectionPool:
    def __init__(self, max_conn=3):
        self.max_conn = max_conn
        self.active_connections = 0

    def acquire_connection(self):
        if self.active_connections >= self.max_conn:
            raise RuntimeError("Maksimum bağlantı sınırına ulaşıldı!")
        self.active_connections += 1
        return f"Conn-{self.active_connections}"

    def release_connection(self):
        # HATA: Bağlantı azaltma mantığı yazılmamış veya eksik bırakılmış (İçerik boş)
        pass

pool = DBConnectionPool()
c1 = pool.acquire_connection()
c2 = pool.acquire_connection()
c3 = pool.acquire_connection()
pool.release_connection()
c4 = pool.acquire_connection() # Limit aşımı hatası fırlatacak
```

FOCUSTRACER DOĞRULAMA RAPORU VE BEKLENTİ:

release_connection metodu gövdesi boş bırakıldığı için (pass) aktif bağlantı sayısı asla düşmez. Döngüsel çağrılarda veya ardışık işlemlerde havuz kilitlenir. Trace loglarında nesne durumunun (state) güncellenmediği gözlemlenebilir.

Senaryo 6: Müşteri İndirim Kuponu Eşleştirmesi

SEMANTİK HATA / TIP GÜVENLİĞİ (TYPE ERROR)

```
def apply_coupon(price, coupon_code):  
    # HATA: Giriş parametre tipi kontrol edilmeden string operasyonu yapılıyor  
    # coupon_code tamsayı (int) gelirse .strip() metodu çökecektir.  
    clean_code = coupon_code.strip().upper()  
    if clean_code == "WINTER20":  
        return price * 0.8  
    return price  
  
# Çağrı tarafında integer gönderiliyor  
print(apply_coupon(200, 2026))
```



FOCUSTRACER DOĞRULAMA RAPORU VE BEKLENTİ:

coupon_code.strip() çağrısı tamsayı nesneleri için geçersizdir ve AttributeError fırlatır. FocusTracer izleme loglarında parametre tipleri ve fırlatılan hata izi (stack trace) takip edilmelidir.

Senaryo 7: Faktöriyel Hesaplama (Özyinelemeli)

MANTIKSAL / SONSUZ DÖNGÜ HATASI (STACK OVERFLOW)

```
def recursive_factorial(n):  
    # HATA: Taban durumu (Base case) yanlış tanımlanmış ya da eksik.  
    # n == 1 yerine n == 0 unutulmuş ve negatif sayılar veya hatalı azaltma durumunda  
    # sonsuz döngü.  
    if n == 1:  
        return 1  
    return n * recursive_factorial(n - 1)  
  
# Negatif veya 0 parametre verilmesi durumunda:  
print(recursive_factorial(0))
```



FOCUSTRACER DOĞRULAMA RAPORU VE BEKLENTİ:

recursive_factorial(0) çağrıldığında taban durumu (`n==1`) asla yakalanamayacağı için `0, -1, -2...` şeklinde sonsuza gider ve RecursionError fırlatır. Trace logunda aynı fonksiyonun üst üste yüzlerce kez çağrıldığı derinlik haritası çıkarılmalıdır.

Senaryo 8: Sensör Verisi Loglayıcı

HATASIZ (CLEAN CODE / BASELINE)

```
import time

def log_sensor_readings(device_id, samples):
    log_entries = []
    for i, val in enumerate(samples):
        entry = {
            "timestamp": time.time(),
            "device": device_id,
            "sequence": i,
            "reading": val
        }
        log_entries.append(entry)
    return log_entries

readings = [14.2, 15.1, 14.9]
logs = log_sensor_readings("SENSOR-A", readings)
print(f"Toplam log: {len(logs)}")
```



FOCUSTRACER DOĞRULAMA RAPORU VE BEKLENTİ:

Döngü ve sözlük oluşturma adımları hatasız çalışır. FocusTracer'ın zaman tabanlı metrikleri ve liste ekleme (append) işlemlerini yapısal XML/JSON formatına kusursuz dönüştürmesi beklenir.

Senaryo 9: Kullanıcı Kayıt Sistemi Şifre Kontrolü

MANTIKSAL HATA (LOGICAL BUG)

```
def validate_registration(username, password):  
    if len(username) < 5:  
        return "Geçersiz Kullanıcı Adı"  
  
    # HATA: Şifre uzunluğu en az 8 olmalıyken mantıksal operatör hatası yapılmış.  
    # length < 8 yerine > 8 yazıldığı için kısa şifreleri kabul edip uzun şifreleri  
    reddediyor.  
    if len(password) > 8:  
        return "Şifre çok kısa!"  
  
    return "Kayıt Başarılı"  
  
print(validate_registration("ahmet_polat", "123"))
```

FOCUSTRACER DOĞRULAMA RAPORU VE BEKLENTİ:

len(password) > 8 kontrolü mantıksal olarak tam tersi çalışmaktadır. Kısa şifrelerde if bloğu atlanır ve kayıt başarılı sayılır. Trace çıktısında koşul doğruluğu izlenmelidir.

Senaryo 10: Dizi Elemanları Farkı Hesaplama

SEMANTİK / İNDEKS HATASI (INDEX ERROR)

```
def calculate_adjacent_differences(numbers):  
    differences = []  
    # HATA: Döngü sınır değer tanımlama hatası (Off-by-one error)  
    # range(len(numbers)) son elemana ulaştığında numbers[i+1] indeks hatası  
    verecektir.  
    for i in range(len(numbers)):  
        diff = numbers[i+1] - numbers[i]  
        differences.append(diff)  
    return differences  
  
print(calculate_adjacent_differences([10, 25, 30, 50]))
```

FOCUSTRACER DOĞRULAMA RAPORU VE BEKLENTİ:

Döngünün son adımında `numbers[i+1]` ifadesi listenin sınırları dışına taşar ve IndexError fırlatır. FocusTracer izleme logunda döngünün hangi iterasyonda çıktığı net olarak yakalanabilir.

Senaryo 11: Ürün Fiyat Güncelleme ve Loglama

SEMANTİK HATA / DEĞİŞKEN KAPSAMI (UNBOUNDLOCALERROR)

```
discount_rate = 0.15

def apply_global_discount(price):
    # HATA: Fonksiyon içinde discount_rate değişkenine atama yapılmaya çalışılıyor.
    # Python bunu local değişken kabul eder ama atamadan önce okumaya çalıştığı için
    # çöker.
    if price > 200:
        price = price - (price * discount_rate)
        discount_rate = 0.20 # Local tanım globali gölgeler
    return price

print(apply_global_discount(250))
```



FOCUSTRACER DOĞRULAMA RAPORU VE BEKLENTİ:

Fonksiyon içinde global anahtar kelimesi kullanılmadan global bir değişkene atama yapılması local kapsam hatasına (UnboundLocalError) yol açar. FocusTracer logları değişken semantiğini yakalamalıdır.

Senaryo 12: Dosya İçeriği Ayırıştırıcı (Parser)

HATASIZ (CLEAN CODE / BASELINE)

```
def parse_config_lines(lines):
    config = {}
    for line in lines:
        cleaned = line.strip()
        if not cleaned or cleaned.startswith("#"):
            continue
        if "=" in cleaned:
            key, val = cleaned.split("=", 1)
            config[key.strip()] = val.strip()
    return config

raw_lines = ["# Ayarlar Dosyası", "port = 8080", "host = localhost", " ", "debug = true"]
print(parse_config_lines(raw_lines))
```



FOCUSTRACER DOĞRULAMA RAPORU VE BEKLENTİ:

Metin ayırıştırma, yorum satırlarını es geçme ve sözlük oluşturma adımları hatasız yürütülür. Kontrol akışlarındaki `continue` durumlarının log doğrulaması yapılabilir.

Senaryo 13: Abonelik Yenileme Tarih Hesaplama

MANTIKSAL / DÖNGÜSEL GÜNCELLEME HATASI

```
def renew_subscription(user_profile):
    # HATA: Boolean ataması yerine yanlışlıkla karşılaştırma operatörü (==)
    kullanılmış.
    # Kullanıcının aktiflik durumu güncellenmez, durum havada kalır.
    if user_profile['has_paid']:
        user_profile['is_active'] == True
        print("Abonelik uzatıldı.")
    return user_profile

profile = {'name': 'Ali', 'has_paid': True, 'is_active': False}
print(renew_subscription(profile))
```

FOCUSTRACER DOĞRULAMA RAPORU VE BEKLENTİ:

user_profile['is_active'] == True satırı bir atama değil karşılaştırmadır. Kod hata vermeden çalışır fakat iş mantığı yerine getirilmez (is_active False kalır). Trace logunda if bloğu yürütülmesine rağmen nesne durumunun değişmediği görülmelidir.

Senaryo 14: İki Listenin Kesişimini Bulma

SEMANTİK / DEĞİŞTİRİLEBİLİR VARSAYILAN ARGÜMAN (MUTABLE DEFAULT ARGUMENT)

```
# HATA: Varsayılan argüman olarak liste ([]) kullanımı.
# Fonksiyon her çağrıldığında eski çağrılardan kalan veriler bu listede birikir!
def find_intersection(list_a, list_b, result_list=[]):
    for item in list_a:
        if item in list_b and item not in result_list:
            result_list.append(item)
    return result_list

res1 = find_intersection([1, 2, 3], [2, 3, 4]) # Beklenen: [2, 3]
res2 = find_intersection([5, 6], [6, 7])      # Beklenen: [6] ama [2, 3, 6] döner!
print("Çağrı 2 Çıktısı:", res2)
```

FOCUSTRACER DOĞRULAMA RAPORU VE BEKLENTİ:

Python'da varsayılan argümanlar fonksiyon tanımında bir kez oluşturulur. İkinci çağrıda `result\_list` sıfırlanmaz. FocusTracer izleme loglarında parametre belleğinin (id/state) çağrılar arasında korunduğu yakalanmalıdır.

Senaryo 15: Önbellek (Cache) Temizleme Mekanizması

SEMANTİK / SÖZLÜK BOYUT DEĞİŞİMİ HATASI

```
def clear_expired_cache(cache_dict):
    # HATA: Sözlük üzerinde döngü dönerken sözlüğün boyutu değiştirilmeye (del)
    # çalışılıyor.
    # Python buna çalışma zamanında izin vermez.
    for key, data in cache_dict.items():
        if data['expired']:
            del cache_dict[key]
    return cache_dict

my_cache = {
    "a": {"expired": False},
    "b": {"expired": True},
    "c": {"expired": True}
}
print(clear_expired_cache(my_cache))
```



FOCUSTRACER DOĞRULAMA RAPORU VE BEKLENTİ:

Döngü esnasında koleksiyon yapısının değiştirilmesi `RuntimeError: dictionary changed size during iteration` hatasına sebep olur. FocusTracer logları döngü kesintisini ve fırlatılan hatayı raporlamalıdır.